Bitácora de uso de IA

Proyecto: UberClone — Versión Conductor. Herramienta usada en todo el proyecto: **Claude**.

Forma de trabajo: para cada parte del taller, Claude listaba los puntos difíciles y el integrante se los preguntaba a un chat de Claude con el prompt de cada entrada. La respuesta de ese chat se usaba como recomendación, y Claude generaba el código a partir de ella. El integrante revisaba el resultado, lo probaba en su iPhone con Expo Go y subía los commits. Los bloques asistidos llevan en el código el comentario `// AI-assisted: see docs/AI_LOG.md #N`.

Lo que va entre **[corchetes]** lo completa el integrante.

Entrada #1

- Fecha: 2026-11-03
- Integrante: [Nombre Apellido]
- Herramienta: Claude
- Objetivo: Planear el taller por fases y crear la lógica de tarifas (pricing.js y calculateFare)
- Prompt: "Necesito que me instrullas para realizar este taller y me des recomendaciones de por donde empiezo"
- Enlace: [enlace a la conversación]
- Resumen de la respuesta: Propuso construir por capas en 10 fases (preparación, esqueleto, tarifas, registro, ubicación, solicitudes, viaje, pagos, ganancias, cierre) y reemplazos compatibles con Expo Go (expo-notifications, expo-location, SDK web de Firebase). Dio el código de `pricing.js` y `calculateFare` con el redondeo a 100 antes de la comisión.
- Qué se usó / modificó / descartó: `pricing.js` y `calculateFare.js` se usaron tal cual. El código del proyecto base lo generó Claude a partir de esta respuesta, y el integrante lo revisó. La primera versión usaba Expo Router; se descartó en la entrada #2.
- Archivos y commit: src/config/pricing.js, src/utils/calculateFare.js, src/utils/formatCOP.js, src/theme/index.js, .env.example — 09ee827
- Verificación: calculateFare dio 7 de 7 casos correctos. Entre ellos están los ejemplos de la sección 5 (8,5 km y 22 min: Económico $19.200 / $3.840 / $15.360; XL $26.900 / $5.380 / $21.520), la tarifa mínima, la propina y una categoría inválida.
- Aprendizaje: [Escríbelo tú. Ideas: por qué se redondea antes de calcular la comisión y por qué se guarda commissionRate en cada viaje.]

Entrada #2

- Fecha: 2026-11-03
- Integrante: [Nombre Apellido]
- Herramienta: Claude
- Objetivo: Armar el esqueleto de navegación (Stack, Tabs y Drawer) y el DriverContext
- Prompt: "[pega aquí el prompt exacto que le hiciste a tu chat sobre la Fase 1]"
- Enlace: [enlace a la conversación]
- Resumen de la respuesta: Recomendó React Navigation con tres navegadores: native-stack para el viaje, bottom-tabs para el registro y drawer para el menú. También recomendó navegación condicional:

el RootNavigator muestra un grupo de pantallas según el estado del conductor, en vez de usar navigate. Dio el DriverContext con useMemo y el hook useDriver.

- Qué se usó / modificó / descartó: El código de la respuesta se usó tal cual. Claude lo aplicó al proyecto, quitó Expo Router y actualizó AGENTS.md para que dijera React Navigation.
- Archivos y commit: App.js, src/context/DriverContext.js, src/navigation/RootNavigator.js, TripStack.js, DrawerNavigator.js, RegistrationTabs.js, src/screens/Placeholder.js — 4aa7cb8
- Verificación: Lint sin errores, expo-doctor 20/20 y compila para Android e iOS. Recorrido completo: registro → Simular registro → Pendiente → Simular aprobación → Drawer → cada sección del menú → Cerrar sesión vuelve al registro.
- Aprendizaje: [Escríbelo tú. Idea: por qué la navegación condicional impide volver con "atrás" a una pantalla que ya no corresponde.]

Entrada #3

- Fecha: 2026-11-03
- Integrante: [Nombre Apellido]
- Herramienta: Claude
- Objetivo: Pestaña 1 del registro (Datos personales) y componentes de formulario
- Prompt: "ok continua con la siguiente fase"
- Resumen de la respuesta: Claude generó la pestaña:
  - RegistrationContext, con un solo estado para las 3 pestañas y los errores derivados con useMemo.
  - Validadores puros: validatePersonal, calculateAge y onlyDigits.
  - Los componentes TextField, SelectField, DateField y PhotoField (cámara o galería con expo-image-picker).
- Qué se usó / modificó / descartó: Se usó lo generado. Las decisiones las tomó Claude y el integrante las revisó:
  - Los errores aparecen al salir del campo o al tocar Siguiente.
  - La edad se calcula comparando año, mes y día.
  - Si se niega el permiso de cámara, sale un aviso debajo del campo.

    El selector de fecha falló en iPhone y se corrigió en la entrada #4.
- Archivos y commit: src/context/RegistrationContext.js, src/utils/registrationValidators.js, src/config/registrationOptions.js, src/components/form/*, src/components/FormScreen.js, src/components/PrimaryButton.js, src/screens/registration/PersonalDataScreen.js, app.json — 1b85d94
- Verificación: 13 de 13 casos del validador: nombre con solo espacios, documento de 5 y de 11 dígitos, alguien que cumple 18 hoy y otro que los cumple mañana, correo sin @. Lint sin errores y expo-doctor 20/20. Probado en iPhone con Expo Go.
- Aprendizaje: [Escríbelo tú. Idea: por qué los errores se calculan con useMemo en vez de guardarse en un useState aparte.]

Entrada #4

- Fecha: 2026-11-03
- Integrante: [Nombre Apellido]
- Herramienta: Claude

- Objetivo: Arreglar el selector de fecha, que no dejaba elegir en iPhone
- Prompt: "[pega aquí el prompt exacto que le enviaste a tu chat con el código de DateField.js]"
- Enlace: [enlace a la conversación]
- Resumen de la respuesta: Explicó que el selector sí se abría, pero quedaba escondido: el teclado phone-pad no se cierra en iPhone y la rueda se dibujaba debajo del campo, tapada. La solución fue cerrar el teclado con Keyboard.dismiss() y mostrar la rueda en un Modal que sube desde abajo, como el SelectField.
- Qué se usó / modificó / descartó: Se usó tal cual. Claude aplicó el cambio en DateField.js y agregó Keyboard.dismiss() en SelectField.js, como recomendó la respuesta.
- Archivos y commit: src/components/form/DateField.js, src/components/form/SelectField.js — 1490e2f
- Verificación: Probado en iPhone. Con el teclado abierto, al tocar la fecha se cierra el teclado y sube la rueda. "Listo" guarda la fecha, y no deja elegir una edad menor de 18 años.
- Aprendizaje: [Escríbelo tú. Idea: qué hace keyboardShouldPersistTaps y por qué en iPhone se usa un Modal.]

Entrada #5

- Fecha: 2026-11-03
- Integrante: [Nombre Apellido]
- Herramienta: Claude
- Objetivo: Pestaña 2 del registro (Vehículo)
- Prompt: "[pega aquí el prompt exacto que le hiciste a tu chat sobre la Pestaña 2]"
- Enlace: [enlace a la conversación]
- Resumen de la respuesta: Recomendó como año mínimo 2001 (requisito de Uber en Colombia) y 13 colores, incluido "Otro". Dio validateVehicle, normalizePlate y las categorías sacadas de PRICING.multipliers, con un mínimo de capacidad por categoría (XL 6, sin contar al conductor). Explicó que el error de capacidad se actualiza solo porque los errores se derivan con useMemo.
- Qué se usó / modificó / descartó: Claude implementó la recomendación. Se modificó la placa: con maxLength 6, "abc-123" quedaba "ABC12" porque el guion ocupaba un espacio. Se cambió a maxLength 7 con normalizePlate(text).slice(0, 6).
- Archivos y commit: src/screens/registration/VehicleScreen.js, src/config/registrationOptions.js, src/utils/registrationValidators.js, src/context/RegistrationContext.js, src/navigation/RegistrationTabs.js, src/components/FormScreen.js, src/components/form/PhotoField.js — 8842998
- Verificación: 15 de 15 casos del validador: placa en minúscula, AB1234, años 2000, 2001 y 2027, XL con 4 y con 6, y cambiar de XL a Económico. "abc-123" queda "ABC123". Con capacidad 4, elegir XL muestra el error y volver a Económico lo quita.
- Aprendizaje: [Escríbelo tú. Idea: por qué la regla de XL va en un objeto de mínimos y no en un if dentro del validador.]

Entrada #6

- Fecha: 2026-11-07
- Integrante: [Nombre Apellido]
- Herramienta: Claude
- Objetivo: Pestaña 3 del registro (Documentos y pagos) y el botón "Enviar registro"

- Prompt:

```
Sigo con la app de Expo SDK 58 (React Navigation, Expo Go en iPhone). Ya tengo las pestañas 1 y 2 del registro con un RegistrationContext que comparte el estado, validadores puros en utils/registrationValidators.js (validatePersonal y validateVehicle) y los componentes TextField, SelectField, DateField (con Modal en iPhone) y PhotoField. Ahora voy con la pestaña 3, Documentos y pagos: Licencia de conducción: número, foto y fecha de vencimiento posterior a hoy. SOAT: foto y fecha de vencimiento posterior a hoy. Revisión técnico-mecánica: foto y fecha de vencimiento, si aplica según el año del vehículo de la pestaña 2. Correo de la cuenta de Mercado Pago, con validación de formato. Checkbox obligatorio de términos con el porcentaje de comisión visible (está en PRICING.commissionRate = 0.2). El botón "Enviar registro" solo se habilita cuando las 3 pestañas son válidas, y por ahora al enviar solo hago setDriver({ status: 'pending' }). ¿Cómo hago validateDocuments? ¿Cómo comparo la fecha de vencimiento contra hoy sin que me afecte la hora? ¿Cuál es la regla en Colombia para saber si un carro particular necesita revisión técnico-mecánica según su año? ¿Cómo valido el número de la licencia? ¿Cómo hago el checkbox? ¿Y cómo le muestro al conductor qué pestaña le falta si el botón está deshabilitado?
```

- Enlace: [enlace a la conversación]
- Resumen de la respuesta:
  - Revisión técnico-mecánica: a los 6 años para los carros matriculados antes del 19-may-2023 (Decreto 019 de 2012) y a los 5 años para los de después (Ley 2294 de 2023), aproximando con el año del modelo.
  - La licencia lleva el número de la cédula.
  - Las fechas se comparan como número AAAAMMDD para ignorar la hora.
  - Dio un CheckboxField con Pressable e Ionicons y una lista tocable de pestañas pendientes encima del botón.
- Qué se usó / modificó / descartó: Claude implementó la recomendación con dos cambios:
  - El mensaje de las fotos dice "del SOAT" en vez de "de el SOAT".
  - La casilla pasó de `accessibilityState` a `role` y `aria-checked`, porque en la prueba no se publicaba si estaba marcada.
- Archivos y commit: src/components/form/CheckboxField.js, src/config/registrationOptions.js, src/context/RegistrationContext.js, src/navigation/RegistrationTabs.js, src/screens/registration/DocumentsScreen.js, src/utils/registrationValidators.js — 2cdca14
- Verificación: 24 de 24 casos de los validadores:
  - Un SOAT que vence hoy cuenta como vencido.
  - La revisión aplica al modelo 2020 y no al 2021 ni al 2024 en 2026.
  - Una licencia distinta a la cédula da error.

    Lint sin errores, la app compila para iOS y Android, y en el navegador funcionan la lista de pestañas pendientes y los campos de la revisión según el año.
- Aprendizaje: [Escríbelo tú. Idea: por qué se compara la fecha como AAAAMMDD y no con getTime().]

Entrada #7

- Fecha: 2026-11-04
- Integrante: [Nombre Apellido]
- Herramienta: Claude
- Objetivo: Conectar la app con Firestore usando variables de entorno

- Prompt: "oe, y ya esta configurada la base de Datos? solo digame y me explica como la configuro para que los datos que yo registre ahi queden guardados en ella, no puedes tener acceso a la base de Datos"
- Resumen de la respuesta: Claude explicó que la base de datos no estaba conectada y guió la creación de Firestore en modo de prueba, del upload preset sin firma de Cloudinary y del .env. Después reescribió configFireBase.js para que lea las variables EXPO_PUBLIC_, exporte db con getFirestore y no use Analytics, que no funciona en Expo Go.
- Qué se usó / modificó / descartó: Se usó lo generado. Se descartó el snippet original de la consola de Firebase, que tenía las llaves escritas en el código y llamaba a getAnalytics.
- Archivos y commit: src/config/configFireBase.js — 8842998
- Verificación: Lint sin errores y Expo carga las 10 variables del .env. Prueba de conexión: se guardó y se leyó el documento pruebas/conexion en Firestore, y se vio en la consola de Firebase.
- Aprendizaje: [Escríbelo tú. Ideas: por qué las llaves van en .env y por qué Firestore guarda solo la URL de la foto.]

Entrada #8

- Fecha: 2026-11-07
- Integrante: [Nombre Apellido]
- Herramienta: Claude
- Objetivo: Enviar el registro: subir las fotos a Cloudinary y guardar al conductor en Firestore
- Prompt:

```
Sigo con la app de Expo SDK 58 (React Navigation, Expo Go en iPhone). Ya tengo las 3 pestañas del registro con validación en un RegistrationContext, y el botón "Enviar registro" se habilita cuando todo es válido. Ya tengo Firestore conectado (`export const db = getFirestore(app)` en `src/config/configFireBase.js`) y un upload preset sin firma de Cloudinary, con `EXPO_PUBLIC_CLOUDINARY_CLOUD_NAME` y `EXPO_PUBLIC_CLOUDINARY_UPLOAD_PRESET` en el `.env`. Las fotos son URIs `file://` de expo-image-picker: perfil, vehículo, licencia, SOAT y revisión técnico-mecánica (esta última solo si aplica). Al tocar "Enviar registro" quiero subir las fotos a Cloudinary, guardar el conductor en la colección `drivers` de Firestore con `status: 'pending'` y después hacer `setDriver`. ¿Cómo subo una foto desde React Native a Cloudinary con fetch y FormData? ¿Subo las fotos en paralelo o una por una, y qué hago si una falla? ¿Qué ID le pongo al documento del conductor? ¿Cómo evito que se envíe dos veces si toca el botón rápido? ¿Cómo muestro el error si falla el internet o Firestore?
```

- Enlace: [enlace a la conversación]
- Resumen de la respuesta: Recomendó:
  - `cloudinary.js` con FormData y un tiempo límite con AbortController.
  - Subir las fotos en paralelo con Promise.allSettled, para saber cuáles fallaron.
  - Un ID creado una sola vez con `doc(collection(db, 'drivers')).id`, para que reintentar no duplique conductores.
  - `withTimeout`, porque Firestore sin conexión no falla, solo queda en cola.
  - Un candado con useRef contra el doble toque.
  - Mensajes de error por tipo de fallo.
- Qué se usó / modificó / descartó: Claude implementó la recomendación con tres cambios:
  - El FormData con `{ uri, name, type }` no funciona con el fetch de Expo SDK 58: lanza "Unsupported FormDataPart". Se cambió por `new File(uri)` de expo-file-system.

    - El campo de la foto de perfil se llama `profilePhoto` y no `photo`.
    - Se agregó un caché por foto, para que un reintento solo vuelva a subir las que fallaron.
- Archivos y commit: src/services/cloudinary.js, src/services/driverRegistration.js, src/screens/registration/DocumentsScreen.js, package.json — 4248ef9
- Verificación: 29 de 29 pruebas con Cloudinary y Firestore simulados:
    - Una foto que falla dice cuál fue.
    - El reintento solo sube las fallidas.
    - No llegan `file://` ni `undefined` a Firestore.
    - La revisión queda en null para un modelo 2022.
    - El tiempo límite es de 15 s.

      Lint, bundles de iOS y Android y expo-doctor 20/20. Envío real probado en iPhone.
- Aprendizaje: [Escríbelo tú. Idea: por qué allSettled y no Promise.all, y por qué el ID se crea antes de subir las fotos.]

Entrada #9

- Fecha: 2026-11-10
- Integrante: [Nombre Apellido]
- Herramienta: Claude
- Objetivo: Guardar la sesión, escuchar el estado de la cuenta en tiempo real y mostrar la pantalla de Pendiente/Rechazado
- Prompt:

```
Sigo con la app de Expo SDK 58 (React Navigation 7, Firebase JS SDK, pruebo en Expo Go). Ya terminé el registro: al tocar "Enviar registro" subo las fotos a Cloudinary, guardo al conductor en Firestore en drivers/{id} con status "pending" (el id lo genero una vez con doc(collection(db, 'drivers')).id) y después hago setDriver({ id, status: 'pending' }) en un DriverContext. El RootNavigator muestra el Registro si driver es null, Pendiente si status es "pending" y el Drawer principal en otro caso.

El PDF dice: "Al enviar, la cuenta queda en estado Pendiente de aprobación y el conductor no puede ponerse Disponible hasta que el estado cambie a Aprobado (puede simularse cambiando el campo en la base de datos)". También dice: "AsyncStorage solo para preferencias y la sesión, nunca para datos sensibles".

Problema: si cierro Expo Go, driver vuelve a null y me sale el registro otra vez.

Explícame y recomiéndame cómo hacer esto:
1. Guardar la sesión en AsyncStorage: ¿qué guardo exactamente (solo el id, o también el último status)? ¿Lo guardo antes o después del setDriver?
2. Leer la sesión al abrir la app: ¿cómo evito que se vea el registro un instante mientras AsyncStorage responde?
3. Escuchar el status en tiempo real con onSnapshot, para que la app pase sola al Drawer cuando lo cambie a "approved" en la consola de Firebase. ¿Pongo el onSnapshot en el DriverContext o en la pantalla de Pendiente? ¿Cómo lo limpio en el useEffect?
4. ¿Qué hago si abro la app sin internet, si el documento ya no existe porque lo borraron en la consola, o si el status es "rejected"?
5. ¿Cómo debe verse la pantalla de Pendiente? ¿Debe tener un botón para cerrar sesión o empezar de nuevo?

Dame el código en JavaScript y explícame lo que me pueden preguntar en la sustentación.
```

- Enlace: [enlace a la conversación]
- Resumen de la respuesta: Recomendó:
  - Guardar en AsyncStorage solo `{ id, status }`, primero en el disco y después en la pantalla.
  - Un estado `restoring` con una pantalla de carga.
  - El onSnapshot en el DriverContext, con `includeMetadataChanges` y `fromCache` para distinguir "sin conexión" de "borrado".
  - Una lista blanca en el RootNavigator: solo `approved` entra.
  - Una pantalla de estado para pendiente, rechazado y desconocido.
- Qué se usó / modificó / descartó: Claude implementó la recomendación con estos cambios:
  - Un `sessionIdRef` evita que un snapshot que llega mientras se cierra la sesión la vuelva a guardar.
  - La pantalla de estado no falla si el conductor queda en null durante el cambio.
  - Después de probar en el iPhone se vio que "Cancelar y empezar de nuevo" solo borraba la sesión del teléfono y el registro seguía en la base de datos. Ahora borra el documento con deleteDoc antes de cerrar la sesión, y sin internet muestra un error.
- Archivos y commit: src/context/DriverContext.js, src/navigation/RootNavigator.js, src/screens/SplashScreen.js, src/screens/AccountStatusScreen.js, src/screens/registration/PersonalDataScreen.js, src/services/driverRegistration.js — 4248ef9 y 33a9de2
- Verificación: En el navegador, con un proyecto de Firebase falso, se probaron estos casos: sin sesión, pendiente sin conexión, rechazado, status mal escrito, aprobado y sesión dañada. También pasaron el lint y los bundles de iOS y Android. En el iPhone: aprobación y rechazo desde la consola, y cancelar la solicitud.
- Aprendizaje: [Escríbelo tú. Idea: por qué el useEffect depende de driver?.id y no de driver.]

Entrada #10

- Fecha: 2026-11-11
- Integrante: [Nombre Apellido]
- Herramienta: Claude
- Objetivo: Fase 4 — Inicio con mapa, interruptor Disponible / No disponible y ubicación en tiempo real
- Prompt:

```
Sigo con la app de Expo SDK 58 (React Navigation 7, Firebase JS SDK, pruebo en Expo Go en un iPhone). Ya terminé el registro: el conductor queda guardado en Firestore en drivers/{id}, y un DriverContext guarda la sesión ({ id, status }) en AsyncStorage y escucha el status con onSnapshot. Solo un conductor con status "approved" entra al Drawer. La primera pantalla del Stack del viaje se llama Home y hoy es un placeholder. Todavía no tengo instalados react-native-maps ni expo-location.

Ahora voy con la Fase 4. El PDF dice: "Inicio: mapa con la ubicación actual y un interruptor Disponible / No disponible. Mientras está Disponible, la ubicación se actualiza en la base de datos cada pocos segundos; al pasar a No disponible o salir de la pantalla, el seguimiento se detiene (función de limpieza de useEffect)". También pide un hook personalizado useDriverLocation, useState para la disponibilidad y AsyncStorage solo para preferencias y "último estado de disponibilidad".

Explícame y recomiéndame cómo hacer esto:
1. El hook useDriverLocation con expo-location: los permisos (solo mientras se usa la app), qué hago si los niega, y cómo limpio bien watchPositionAsync en el useEffect si el usuario sale de la pantalla antes de que la promesa responda.
```

```
2. Cada cuánto guardar la ubicación en Firestore para no gastar el cupo gratis (timeInterval, distanceInterval o un throttle), y dónde guardarla: en el mismo documento drivers/{id} o en otra colección.
3. Qué guardo al pasar a No disponible, y cómo evito que quede un conductor "fantasma" disponible si la app se cierra de golpe.
4. El mapa con react-native-maps en Expo Go en iPhone: ¿funciona Google Maps o tengo que usar Apple Maps? ¿Necesito una API key para probar?
5. Si guardo el último estado de disponibilidad en AsyncStorage, ¿al abrir la app lo restauro como Disponible automáticamente o no?

Dame el código en JavaScript y explícame lo que me pueden preguntar en la sustentación.
```

- Enlace: [enlace a la conversación]
- Resumen de la respuesta: Recomendó:
  - useDriverLocation con expo-location, una bandera `cancelled` para limpiar watchPositionAsync aunque la promesa llegue tarde, y un botón "Abrir Ajustes" si el permiso queda bloqueado.
  - Escribir máximo cada 10 s si el conductor se movió 30 m, más un latido por minuto, en una colección aparte `driverLocations`.
  - Usar `useIsFocused` para detener el seguimiento al salir de la pantalla.
  - Apple Maps en iPhone sin API key.
  - No restaurar Disponible automáticamente.
- Qué se usó / modificó / descartó: Claude generó el código a partir de la recomendación con estos cambios:
  - Se quitaron los setState dentro de efectos, que rechazaba el lint de Expo.
  - No se publica una posición vieja al reconectarse.
  - La limpieza del conductor "fantasma" no pisa lo que el conductor elige al abrir.
  - El botón de centrar va encima de la tarjeta, no a una altura fija.
  - Se muestran los errores de ubicación y de Firestore.
- Archivos y commit: src/hooks/useDriverLocation.js, src/hooks/useLocationPublisher.js, src/services/driverLocation.js, src/utils/geo.js, src/screens/trip/HomeScreen.js, src/navigation/TripStack.js, app.json — 9fcc760
- Verificación: npx expo lint, bundles de iOS y Android con npx expo export, 10 de 10 pruebas de la distancia y de la regla de envío, y prueba en el iPhone revisando driverLocations en la consola de Firebase.
- Aprendizaje: [Escríbelo tú. Idea: por qué abrir Historial no desmonta Inicio y por eso se usa useIsFocused.]

Entrada #11

- Fecha: 2026-11-11
- Integrante: [Nombre Apellido]
- Herramienta: Claude
- Objetivo: Fase 5 — solicitud entrante con temporizador de 15 segundos
- Prompt:

```
Sigo con la app del conductor en Expo SDK 58 (React Navigation 7, Firebase JS SDK, pruebo en Expo Go en un iPhone).
```

```
Lo que ya tengo:
- Registro terminado: el conductor está en drivers/{id} con status "approved".
- DriverContext guarda la sesión ({ id, status }) en AsyncStorage y tiene activeTrip /
setActiveTrip, que todavía no se usan.
- Fase 4 lista: Inicio (pantalla Home del Stack del viaje) muestra un mapa de Apple Maps con
react-native-maps y un interruptor Disponible. Mientras está Disponible, publica en
driverLocations/{driverId} { available, location (GeoPoint), heading, updatedAt (serverTimestamp)
}. Escribe como máximo cada 10 s si se movió 30 m, y un latido por minuto. Un conductor cuenta
como disponible si available === true y updatedAt es de los últimos 2 minutos.
- El seguimiento depende de useIsFocused: si Home pierde el foco, el conductor queda No
disponible.
- El Stack del viaje tiene Home, IncomingRequest (presentation: 'modal'), TripInProgress y
TripSummary. Los tres últimos son placeholders.
- Ya existe calculateFare({ distanceKm, durationMin, category, tip }) en
src/utils/calculateFare.js. Devuelve { fare, commission, driverEarnings, commissionRate } con
PRICING en src/config/pricing.js (comisión 20 %, categorías economy / xl / premium).

Ahora voy con la Fase 5. El PDF dice: "Solicitud entrante: tarjeta con origen, destino, distancia,
duración, categoría, método de pago, tarifa estimada y ganancia neta estimada del conductor. Tiene
un temporizador de 15 segundos; si llega a cero, la solicitud se rechaza automáticamente."

Estados del viaje (campo status): requested, accepted ("si rechaza o se vence el tiempo, la
solicitud pasa al siguiente conductor o queda rejected"), driver_arriving, in_progress, completed,
paid, cancelled.

Modelo sugerido en la colección trips: driverId, passengerId, origin y destination { lat, lng,
address }, distanceKm, durationMin (de Distance Matrix), category, fare, commission, tip,
driverEarnings, commissionRate, paymentMethod (mercadopago, stripe, cash), paymentStatus,
paymentId, status, rating, createdAt, completedAt.

El PDF pide useEffect con limpieza para el temporizador y las suscripciones en tiempo real,
useCallback para aceptar y rechazar, y un hook personalizado useTripRequests. También dice que
Distance Matrix se llama una sola vez al crear la solicitud y una vez al aceptarla.

Explícame y recomiéndame:
1. Como no tengo app del pasajero, ¿cómo simulo las solicitudes (por ejemplo, un botón que solo
exista en desarrollo con __DEV__) y cómo decido a qué conductor se le ofrece (por ejemplo, un
campo offeredTo)?
2. El hook useTripRequests: qué consulta escucho con onSnapshot, cómo la limpio y si necesito un
índice en Firestore.
3. El temporizador de 15 segundos: ¿cuento desde la hora del servidor o desde que llega al
teléfono? ¿Cómo lo limpio? ¿Qué escribo en la base de datos cuando se vence o cuando rechazo, si
no tengo backend que pase la solicitud al siguiente conductor?
4. Aceptar sin carreras: si acepto justo cuando se vence, ¿uso runTransaction? ¿Qué guardo en el
viaje? ¿Cómo uso activeTrip en DriverContext y cómo recupero el viaje si la app se cierra después
de aceptar?
5. ¿Muestro la solicitud en el modal IncomingRequest (Home pierde el foco y quedo No disponible) o
como una tarjeta encima del mapa? ¿Qué es más correcto?
6. Distancia, duración y tarifa: ¿quién llama a Distance Matrix si no hay app del pasajero? ¿Es un
problema tener la API key en la app? ¿Hay una alternativa gratis para la simulación? ¿En qué
momento calculo la tarifa y la ganancia con calculateFare y la guardo en el viaje?
7. ¿Cómo aviso al conductor que llegó una solicitud (vibración o sonido) si en Expo Go no puedo
usar Firebase Cloud Messaging?

Dame el código en JavaScript y explícame lo que me pueden preguntar en la sustentación.
```

- Enlace: [enlace a la conversación]
- Resumen de la respuesta: Recomendó:
  - Simular al pasajero con un botón que solo existe con `__DEV__`.

    - Ofrecer cada viaje a un conductor a la vez con `offeredTo`, y guardar en `declinedBy` quién la rechazó.
    - useTripRequests con una consulta de solo igualdades, que no necesita índice.
    - Un temporizador contra una hora límite, medida con el reloj del teléfono.
    - runTransaction con la misma condición para aceptar y rechazar.
    - El viaje activo guardado como puntero en AsyncStorage.
    - Una tarjeta sobre el mapa en vez del modal.
    - OSRM, porque Google pasó Directions y Distance Matrix a Legacy en 2025.
    - Vibración con expo-haptics.
- Qué se usó / modificó / descartó: Claude generó el código a partir de la recomendación con estos cambios:
    - El temporizador no llama setState dentro del efecto, porque lo rechazaba el lint de Expo.
    - La hora de llegada de la oferta va en el estado y no en un ref leído al renderizar.
    - La ruta a la recogida la calcula la pantalla del viaje, así también funciona si la app se reinicia justo después de aceptar.
    - Las rutas usan primero la Routes API de Google (el reemplazo oficial) y después OSRM.

      Se descartó la pantalla modal IncomingRequest.
- Archivos y commit: src/hooks/useTripRequests.js, src/components/IncomingRequestCard.js, src/services/trips.js, src/services/devTripSimulator.js, src/services/routing.js, src/screens/trip/HomeScreen.js, src/context/DriverContext.js, src/navigation/TripStack.js — 16bc413
- Verificación: Pruebas con un Firestore simulado: aceptar, rechazar, vencer, aceptar después del vencimiento, una oferta de otro conductor y una solicitud cancelada. Rutas probadas con fetch simulado para Google, OSRM y sin red. Lint, bundles de iOS y Android y prueba en iPhone.
- Aprendizaje: [Escríbelo tú. Idea: por qué una transacción garantiza que, entre aceptar y vencer, gane exactamente una.]

Entrada #12

- Fecha: 2026-11-11
- Integrante: [Nombre Apellido]
- Herramienta: Claude
- Objetivo: Fase 6 — viaje en curso (ruta a la recogida, pasajero a bordo, ruta al destino y finalizar)
- Prompt: "realiza todo lo que falte para terminar el taller"
- Resumen de la respuesta: Claude implementó:
    - La pantalla del viaje, con la ruta dibujada con Polyline y un botón para abrir Apple Maps.
    - Las transiciones del viaje con runTransaction: driver_arriving, in_progress, completed y cancelled.
    - La tarifa final con calculateFare, usando la ruta real, los minutos reales y el commissionRate guardado en el viaje.
    - Un mapa de reemplazo para web.
- Qué se usó / modificó / descartó: Se usó tal cual.
- Archivos y commit: src/screens/trip/TripScreen.js, src/services/trips.js, src/utils/calculateFare.js, src/utils/geo.js, src/components/Map.js, src/components/Map.web.js — 16bc413
- Verificación: Pruebas del cierre del viaje (ruta real, minutos reales, comisión guardada y propina), decodificación de la polyline con el ejemplo oficial de Google, y comprobación de que no se puede

cancelar con el pasajero a bordo. Prueba en iPhone.
- Aprendizaje: [Escríbelo tú. Idea: por qué la tarifa final usa el commissionRate guardado en el viaje y no el de pricing.js.]

Entrada #13

- Fecha: 2026-11-11
- Integrante: [Nombre Apellido]
- Herramienta: Claude
- Objetivo: Fase 7 — cobro con Mercado Pago Checkout Pro (backend) y pago en efectivo
- Prompt: "realiza todo lo que falte para terminar el taller"
- Resumen de la respuesta: Claude creó un backend con Express:
  - Lee el monto del viaje en Firestore, no lo recibe de la app.
  - Crea la preferencia de Checkout Pro con el access_token, que solo vive en el backend.
  - Consulta el pago en Mercado Pago, comprueba el monto y marca el viaje como pagado.
  - Incluye un webhook que valida la firma x-signature.

    En la app hizo la pantalla de cobro, con propina, pago en efectivo y calificación del pasajero. Después se simplificó la puesta en marcha: el backend usa la configuración web de Firebase si no hay cuenta de servicio, y la app encuentra sola la dirección del backend en desarrollo.
- Qué se usó / modificó / descartó: Se usó tal cual.
- Archivos y commit: backend/, src/services/payments.js, src/screens/trip/TripSummaryScreen.js — 16bc413
- Verificación: 12 pruebas con node --test: monto, viaje sin terminar o ya pagado, pago aprobado, rechazado o por otro monto, webhook y firma. El backend arrancó y respondió en /health. Pago de prueba con la tarjeta APRO en iPhone.
- Aprendizaje: [Escríbelo tú. Idea: por qué la app manda solo el tripId y no el monto.]

Entrada #14

- Fecha: 2026-11-11
- Integrante: [Nombre Apellido]
- Herramienta: Claude
- Objetivo: Ganancias (hoy, semana, mes y saldo) e historial con filtros
- Prompt: "realiza todo lo que falte para terminar el taller"
- Resumen de la respuesta: Claude hizo:
  - useEarnings, con onSnapshot y useMemo. El saldo es la ganancia de los viajes pagados por la pasarela menos las comisiones de los viajes en efectivo.
  - useTripHistory, paginado de a 20 con un índice compuesto, y filtros Hoy, Semana, Mes, Rango y método de pago.
  - El detalle del viaje con la ruta dibujada.
- Qué se usó / modificó / descartó: Se usó tal cual.
- Archivos y commit: src/hooks/useEarnings.js, src/hooks/useTripHistory.js, src/utils/earnings.js, src/utils/dates.js, src/screens/EarningsScreen.js, src/screens/history/, src/navigation/HistoryStack.js, src/components/Chip.js, firestore.indexes.json — 16bc413

- Verificación: Pruebas de los totales por periodo, el saldo negativo, la semana que empieza el lunes y los rangos de fechas. Revisión en el navegador de las pantallas de Ganancias e Historial. Prueba en iPhone.
- Aprendizaje: [Escríbelo tú. Idea: por qué el historial necesita un índice compuesto y las ganancias no.]

Entrada #15

- Fecha: 2026-11-11
- Integrante: [Nombre Apellido]
- Herramienta: Claude
- Objetivo: Perfil, configuración (idioma y cierre de sesión), ingreso y README
- Prompt: "realiza todo lo que falte para terminar el taller"
- Resumen de la respuesta: Claude hizo:
  - El perfil, con los datos de contacto editables.
  - La configuración, con el idioma español o inglés guardado en AsyncStorage y el cierre de sesión.
  - La pantalla Ingresar, con documento y celular.
  - El README, con la instalación, las variables de entorno y la sección Uso de IA.
- Qué se usó / modificó / descartó: Se usó tal cual.
- Archivos y commit: src/screens/ProfileScreen.js, src/screens/SettingsScreen.js, src/screens/LoginScreen.js, src/i18n/, src/services/driverAccount.js, src/navigation/, App.js, README.md — 16bc413
- Verificación: Una prueba de que cada clave de traducción existe en español y en inglés. En el navegador, con un Firebase falso: Ingresar, el menú y el cambio de idioma. Prueba en iPhone.
- Aprendizaje: [Escríbelo tú. Idea: por qué sin Firebase Auth el ingreso con documento y celular no es seguro.]